



CSE461

Introduction to Robotics Lab

LAB REPORT - 1

Group : 3

Section : 9

Semester : Summer_2026

Group members :

M. Aliul Siddique Antor	21301633
Mihir Das	22299480
Fariaa Afroz Esha	22201530
Farhan Zarif	23301692
Hirok Roy Rahul	22299082

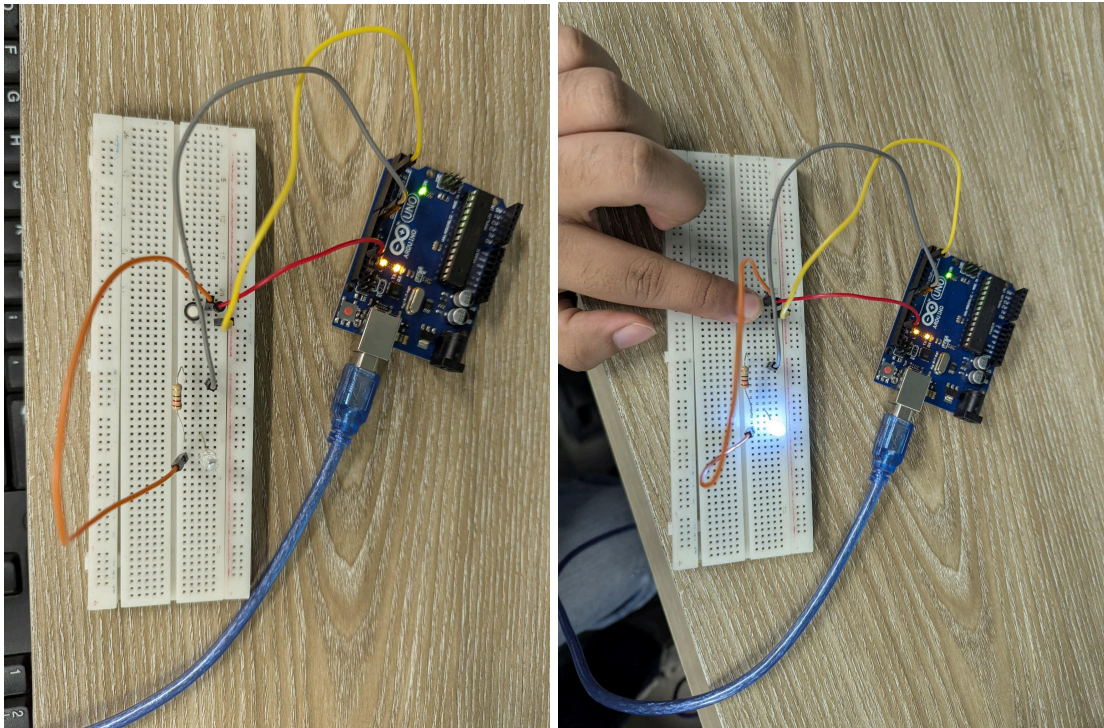
Submitted Date: 25-07-2026

Submitted to - Maheer Helal, Maharin Mosfakin

Lab 1

1. **Equipments:** Arduino Uno R3, LED, Push-button switch, Resistor (220Ω), Breadboard, Jumper wires

2. **Experimental Setup:** (Picture)



3. **Code:**

```
# Enter Code Here
#define BUTTON_PIN 2 // Define the button pin
#define LED_PIN 8

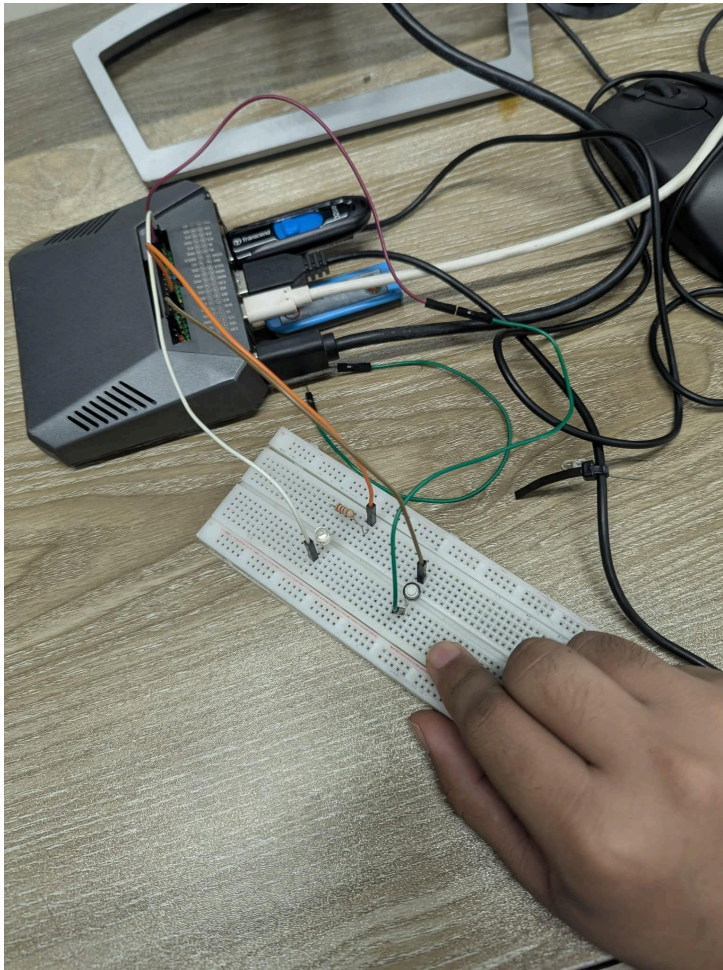
void setup() {
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  pinMode(LED_PIN, OUTPUT); // Set the button pin as input
  Serial.begin(9600);        // Initialize serial communication
}

void loop() {
  if (digitalRead(BUTTON_PIN) == LOW) { // Button is pressed
    digitalWrite(LED_PIN, HIGH); // Turn the LED on
    Serial.println("LED ON");
    Serial.println("Button pressed."); // Print message to S. Monitor
  } else {
    Serial.println("Button not pressed.");
    digitalWrite(LED_PIN, LOW); // Turn the LED off
  }
}
```

```
Serial.println("LED OFF"); // Print LED status
}
delay(100); // Small delay to debounce the button
}
```

Lab 2

4. **Equipments:** Raspberry Pi development board, LED, Resistor (220Ω), Switch, USB pen drive with OS
5. **Experimental Setup:** (Picture)



6. Code:

```
# Enter Code Here

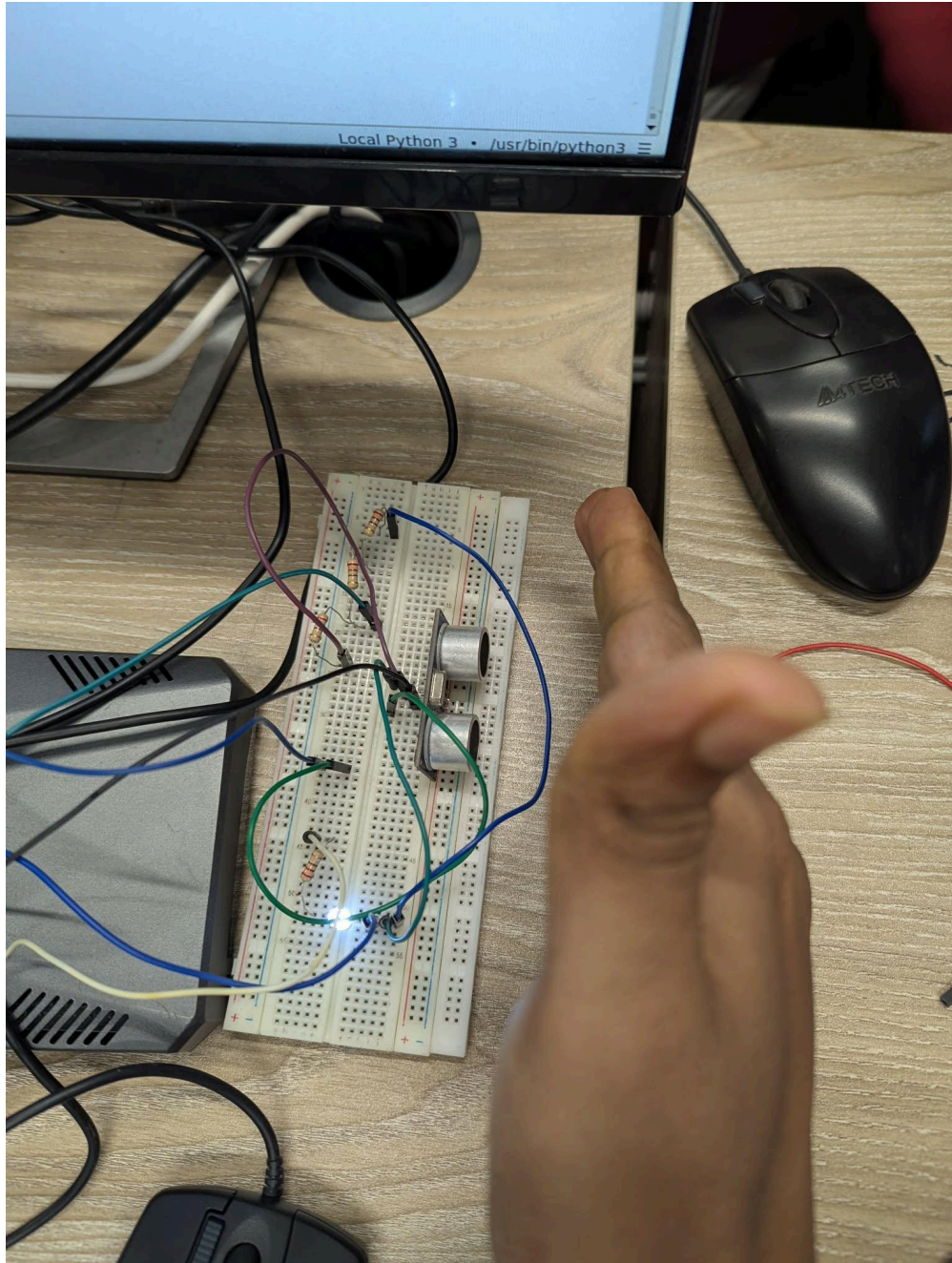
from gpiozero import LED, Button
import time

led = LED(27)
button = Button(16)
while True:
    if button.is_pressed:
        print("Switch pressed")
        led.on()
    else:
        print("Not pressed")
        led.off()

    time.sleep(0.5)
```

Lab 3

1. **Equipments:** Raspberry Pi development board, LED, Ultrasonic Sensor (HC-SR04), Resistor (220Ω), Jumper wires
2. **Experimental Setup:** (Picture)



3. Code:

Enter Code Here

```
import RPi.GPIO as GPIO
import time
GPIO.setmode(GPIO.BCM)
TRIG = 21
```

```
ECHO = 20
LED = 16
GPIO.setup(TRIG,GPIO.OUT)
GPIO.setup(ECHO,GPIO.IN)
GPIO.setup(LED,GPIO.OUT)

def distance():
    GPIO.output(TRIG, False)
    time.sleep(0.5)
    GPIO.output(TRIG, True)
    time.sleep(0.00001)
    GPIO.output(TRIG, False)
    pulse_start = time.time()
    while GPIO.input(ECHO)==0:
        pulse_start = time.time()
    while GPIO.input(ECHO)==1:
        pulse_end = time.time()
    pulse_duration = pulse_end - pulse_start
    distance = pulse_duration * ((343*100)/2)
    distance = round(distance, 2)

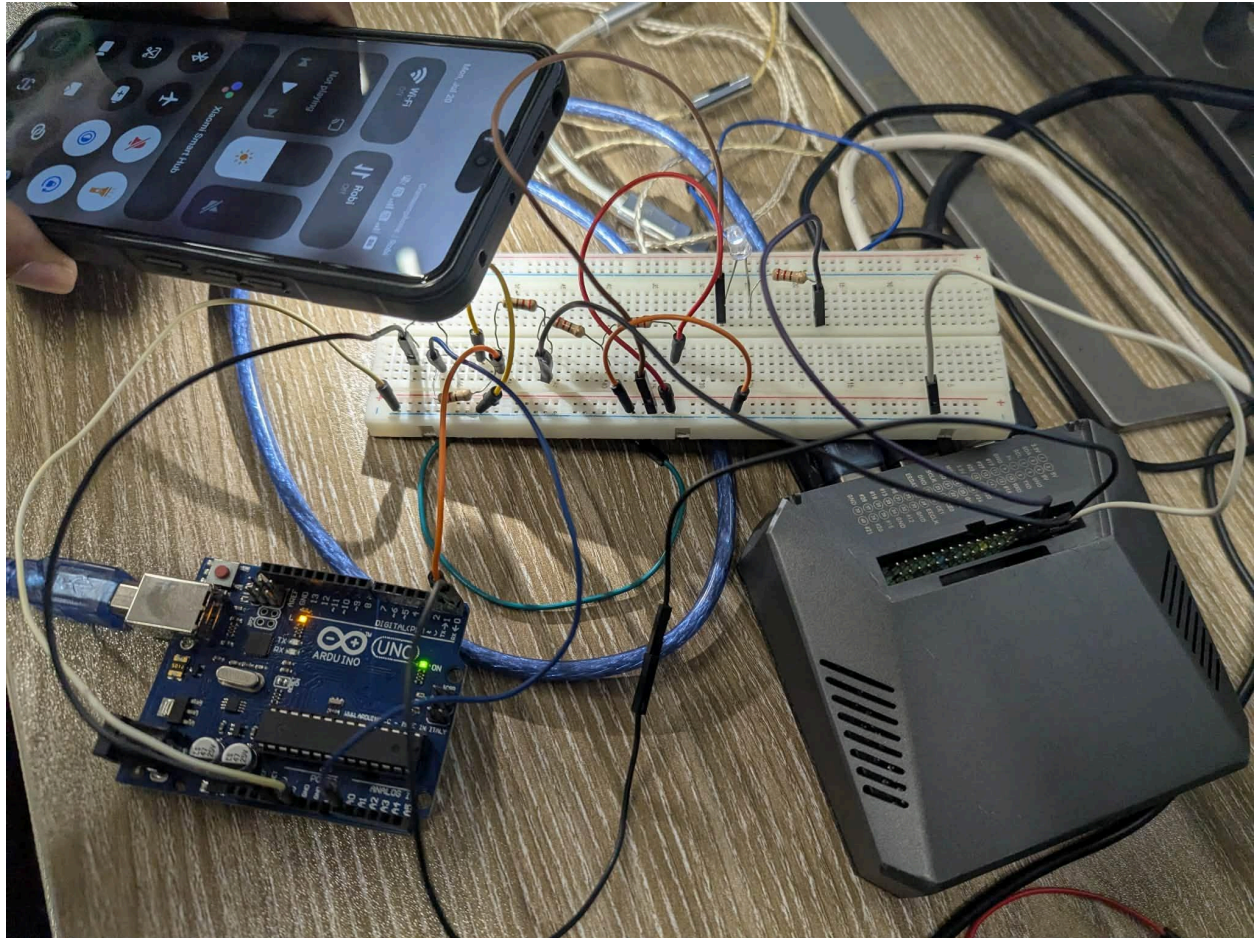
    return distance

while True:
    dist = distance()
    print(f"Distance: {dist}")

    if dist<10:
        GPIO.output(LED, GPIO.HIGH)
        print(f"LED ON")
    else:
        GPIO.output(LED, GPIO.LOW)
        print(f"LED OFF")
GPIO.cleanup()
```

Lab 4

1. **Equipments:** Arduino Uno R3, Raspberry Pi 4, LED, LDR sensor, Resistor (220Ω), Breadboard, Jumper wires
2. **Experimental Setup:** (Picture)



3. Code:

```
//Arduino Code:

int ldrPin = A0;
int ldrValue = 0;

void setup() {
  Serial.begin(9600);
}

void loop() {
  ldrValue = analogRead(ldrPin); // Read LDR value (0-1023)
  Serial.println(ldrValue); // Send value to Raspberry Pi
  delay(100);
}
```

```
#Raspberry PI Code

import serial
import RPi.GPIO as GPIO
```

```
import time

# Setup
GPIO.setmode(GPIO.BCM)
led_pin = 18
GPIO.setup(led_pin, GPIO.OUT)

# Initialize PWM
pwm_led = GPIO.PWM(led_pin, 1000) # 1 kHz frequency
pwm_led.start(0)

# Connect to Arduino
arduino = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
time.sleep(2)
norm = 2.5

try:
    while True:
        data = arduino.readline().decode('utf-8').strip()
        if data.isdigit():
            ldr_value = int(data)
            brightness = ((1-(ldr_value / 1023)) ** norm) * 100
            Brightness = max(0, min(brightness, 100))
            pwm_led.ChangeDutyCycle(Brightness)
            print(f'LDR: {ldr_value}, LED Brightness: {brightness:.1f}%")
except KeyboardInterrupt:
    print("Exiting...")
finally:
    pwm_led.stop()
    GPIO.cleanup()
    arduino.close()
```

Questions

1. The HC-SR04's ECHO pin outputs 5V, but the Pi's GPIO can only handle 3.3V. Using one 220Ω resistor on the echo side and two 220Ω resistors (in series) to ground, calculate the voltage that actually reaches the GPIO pin. Show your calculation, and explain in one or two sentences why this arrangement protects the Pi. [Lab 3]
2. The button pin is set to `INPUT_PULLUP` and the code checks for `LOW` when the button is pressed. Explain why pressing the button gives LOW instead of HIGH, and what the internal pull-up resistor keeps the pin at when the button is not pressed. [Lab 1, 2]
3. In the distance formula, the echo time is divided by 2 before converting to a distance. Explain why that division by 2 is needed, based on how the sensor sends and receives sound. [Lab 3]
4. In Experiment 1, the LDR connects between 5V and pin A0, with a 220Ω resistor from A0 to ground. The worksheet mentions that bright light lowers the LDR's resistance. Using that fact, predict whether the number returned by `analogRead(A0)` goes **up or down** when a bright light shines on the LDR, and explain why in one or two sentences. [Lab 4]

Answers

1. Voltage divider calculation (Lab 3)

The Echo pin's 5V signal first passes through one 220Ω resistor (in series, toward the GPIO junction), and the GPIO junction is also tied to ground through two 220Ω resistors in series (440Ω total).

$$V_{out} = V_{in} \times \frac{R_{to\ GND}}{R_{Series} + R_{to\ GND}} = 5V \times \frac{440\Omega}{220\Omega + 440\Omega} \approx 3.3V$$

This works out to about 3.33V, just above the Pi's 3.3V rating but safely under its absolute damage threshold.

The voltage divider scales down the 5V Echo signal by a factor of $\frac{440}{660} = \frac{2}{3}$. The top 220Ω resistor drops about 1.67V, leaving a scaled down 3.3V across the bottom 440Ω resistors to reach the GPIO pin instead of exposing it to the full 5V. This is how it protects Pi.

2. INPUT_PULLUP and button logic (Lab 1, 2)

INPUT_PULLUP connects the pin internally to 5V (or 3.3V on the Pi) through a pull up resistor, so when the button is not pressed, the pin floats at HIGH because it's only connected to that pull up voltage and nothing else. When you press the button, it creates a direct path from the pin to GND. Since GND offers a much lower resistance path than the pull up resistor, the pin voltage gets pulled down to LOW. That's why pressing the button reads as LOW instead of HIGH, so the button doesn't add voltage, it shorts the pin to ground.

Not pressed => pin held HIGH by the internal pull up resistor.

3. Dividing echo time by 2 (Lab 3)

The ultrasonic sensor's timer starts when the trigger pulse sends the sound wave out and stops only when the reflected wave returns, so the measured time covers the round trip: sensor => object => back to sensor. Since we only want the one way distance to the object, we divide the total travel time by 2 before multiplying by the speed of sound.

4. LDR and analogRead(A0) behavior (Lab 4)

Here A0 sits at the junction between the LDR (connected to 5V) and the fixed 220Ω resistor (connected to GND) a voltage divider where:

$$V_{A0} = V_{in} \times \frac{R_{fixed}}{R_{fixed} + R_{LDR}}$$

When bright light hits the LDR, its resistance decreases. That shrinks the denominator relative to the fixed resistor, which increases the voltage at A0. Since analogRead() converts that voltage into a proportional digital value (0-1023), the reading goes up under bright light and goes down in the dark.